

HW2_Problem1

January 28, 2026

1 Problem 1

We are going to use different variants of regressors to model a sinusoidal function. Let's create a set of non-linear data:

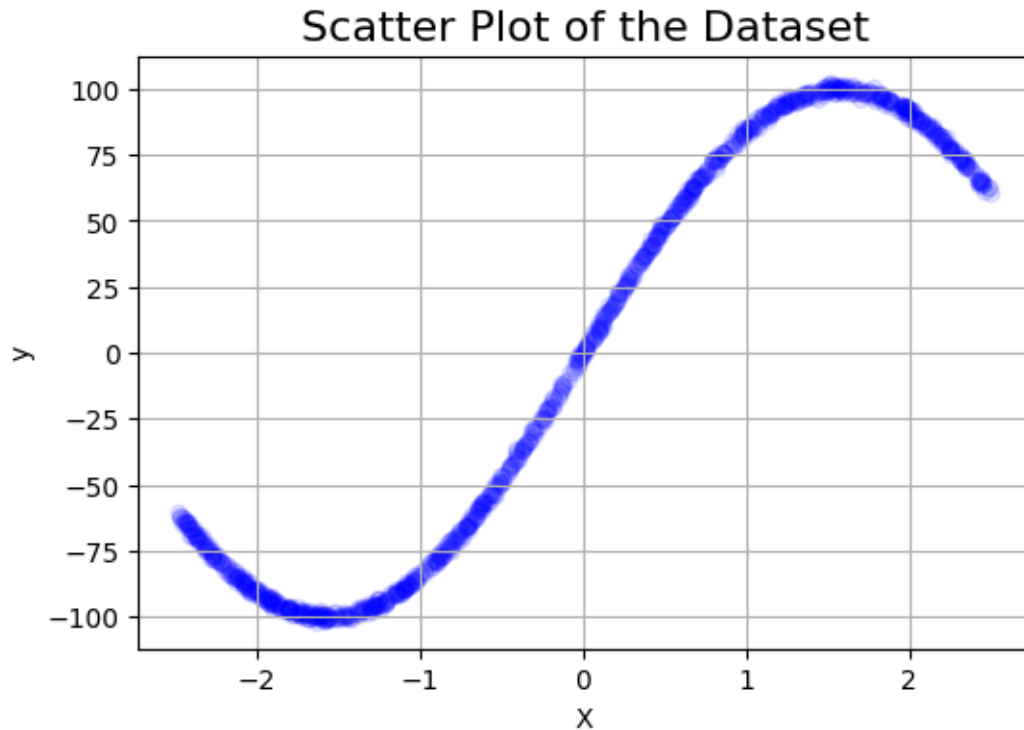
```
[2]: import numpy as np
      np.random.seed(42)
      m = 1000
      X = 5 * np.random.rand(m, 1) - 2.5
      y = np.sin(X)*100 + np.random.randn(m, 1)
```

```
[13]: import matplotlib.pyplot as plt
      from sklearn.linear_model import LinearRegression, Ridge
      from sklearn.preprocessing import StandardScaler, PolynomialFeatures
      from sklearn.pipeline import make_pipeline
      from sklearn.model_selection import learning_curve, cross_val_score
```

Plotting a scatter plot to visualize the distribution of X and y values in the dataset.

```
[3]: import matplotlib.pyplot as plt

      plt.figure(figsize=(6, 4))
      plt.scatter(X, y, color='b', alpha=0.1)
      plt.title("Scatter Plot of the Dataset", fontsize=16)
      plt.xlabel("X")
      plt.ylabel("y")
      plt.grid(True)
      plt.show()
```



1.1 1.1

1.1.1 Apply standardization to the data and plot the learning curve for Linear Regression.

Create a pipeline that first standardizes the data using StandardScaler, and then fits a Linear Regression model.

```
[5]: from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

pipeline_linear = make_pipeline(StandardScaler(), LinearRegression())
pipeline_linear
```

```
[5]: Pipeline(steps=[('standardscaler', StandardScaler()),
                      ('linearregression', LinearRegression())])
```

Compute learning curves using cross-validation (5-fold) and the negative RMSE as the scoring metric.

Use `learning_curve` to compute model performance on varying sizes of the training set.

Parameters:

`train_sizes=np.linspace(0.01, 1.0, 40)` → Generates 40 values from 1% to 100% of the training data.

cv=5 → 5-fold cross-validation.

scoring='neg_root_mean_squared_error' → Returns negative RMSE (sklearn convention)

Learning curves help us understand:

Whether the model suffers from bias (underfitting) or variance (overfitting).

How model performance evolves as more training data becomes available.

```
[16]: train_sizes, train_scores, valid_scores = learning_curve(pipeline_linear, X, y,
    ↪train_sizes=np.linspace(0.01, 1.0, 40),
    cv=5,
    ↪scoring="neg_root_mean_squared_error")
# Negate the scores to get the RMSE for training and validation
train_errors= -train_scores.mean(axis=1)
valid_errors= -valid_scores.mean(axis=1)
```

```
[17]: print(f"Average training error (RMSE): {train_errors.mean():.3f} \nAverage
    ↪validation error (RMSE) : {valid_errors.mean():.3f}")
```

Average training error (RMSE): 27.390

Average validation error (RMSE) : 27.369

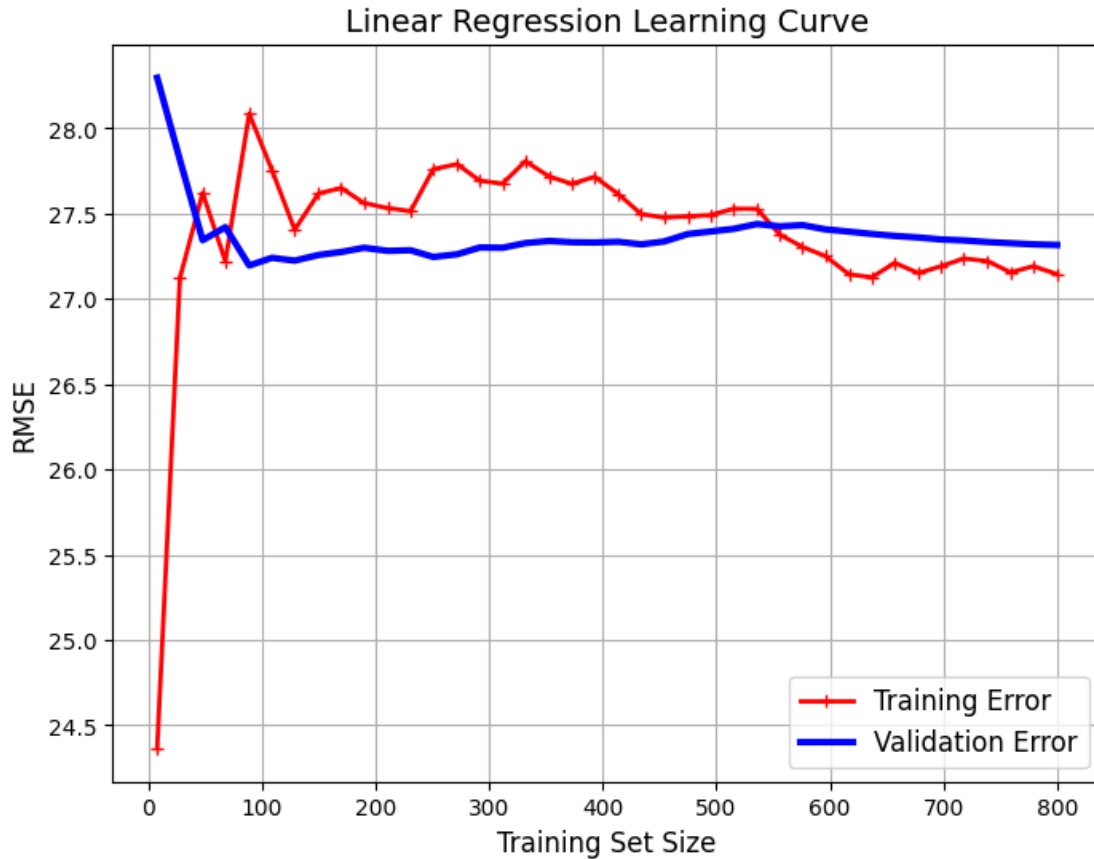
Plot the learning curves for Linear Regression

Small difference between train and validation RMSE indicates:

Low variance (model generalizes similarly to new data).

Bias might still be present if errors are large.

```
[18]: plt.figure(figsize=(8,6))
plt.plot(train_sizes, train_errors, "r--", linewidth=2, label="Training Error")
plt.plot(train_sizes, valid_errors, "b-", linewidth=3, label="Validation Error")
plt.title("Linear Regression Learning Curve", fontsize=14)
plt.xlabel("Training Set Size",fontsize=12)
plt.ylabel("RMSE",fontsize=12)
plt.legend(loc='lower right',fontsize=12)
plt.grid(True)
plt.show()
```



Interpretation:

Training Error Curve (Red):

Starts low (perfect fit on small samples).

Increases as more data is added.

Validation Error Curve (Blue):

Starts high (underfitting with small data).

Stabilizes as more data is added.

Both curves flatten at a high RMSE → Likely underfitting (high bias).

1.2 1.2

1.2.1 Transform the data into a polynomial of degree 50, apply standardization and plot the learning curve.

Create a pipeline for Polynomial Regression that transforms data into polynomial features of degree 50, standardizes the features, and fits a linear model to the polynomial-transformed features.

```
[19]: pipeline_polynomial = make_pipeline(PolynomialFeatures(degree=50,
    ↪include_bias=False), StandardScaler(), LinearRegression())
pipeline_polynomial
```

```
[19]: Pipeline(steps=[('polynomialfeatures',
                        PolynomialFeatures(degree=50, include_bias=False)),
                      ('standardscaler', StandardScaler()),
                      ('linearregression', LinearRegression())])
```

Compute learning curves for the polynomial regression model, using cross-validation (5-fold) and the negative RMSE as the scoring metric.

```
[20]: train_sizes, train_scores, valid_scores = learning_curve(
        pipeline_polynomial, X, y, train_sizes=np.linspace(0.01, 1.0, 40), #
        ↪training sizes range from 1% to 100% of the data in 40 steps.
        cv=5, scoring="neg_root_mean_squared_error")

# Again, negate the scores to get the RMSE for training and validation.
train_errors = -train_scores.mean(axis=1)
valid_errors = -valid_scores.mean(axis=1)
```

```
[21]: print(f"Average training error (RMSE): {train_errors.mean():.3f} \nAverage
        ↪validation error: {valid_errors.mean():.3f}")
```

Average training error (RMSE): 0.845

Average validation error: 317887972.780

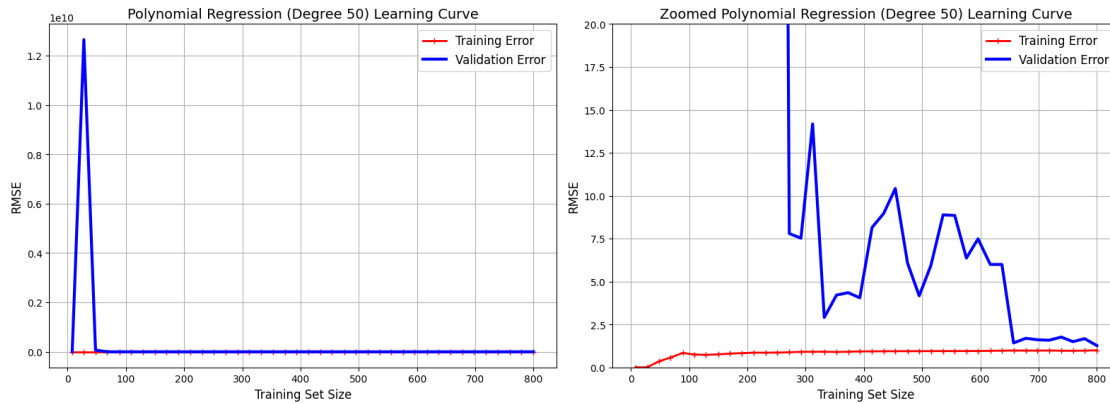
Plot learning curves for polynomial regression (degree 50)

```
[22]: plt.figure(figsize=(16, 6)) # Set figure size for dual subplots

plt.subplot(1, 2, 1)
plt.plot(train_sizes, train_errors, "r-+", linewidth=2, label="Training Error")
plt.plot(train_sizes, valid_errors, "b-", linewidth=3, label="Validation Error")
plt.title("Polynomial Regression (Degree 50) Learning Curve", fontsize=14)
plt.xlabel("Training Set Size", fontsize=12)
plt.ylabel("RMSE", fontsize=12)
plt.legend(loc='upper right', fontsize=12)
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(train_sizes, train_errors, "r-+", linewidth=2, label="Training Error")
plt.plot(train_sizes, valid_errors, "b-", linewidth=3, label="Validation Error")
plt.title("Zoomed Polynomial Regression (Degree 50) Learning Curve",
        ↪fontsize=14)
plt.xlabel("Training Set Size", fontsize=12)
plt.ylabel("RMSE", fontsize=12)
plt.legend(loc='upper right', fontsize=12)
plt.grid(True)
plt.ylim(0, 20)

plt.tight_layout()
plt.show()
```



1.3 1.3

1.3.1 Repeat the process for a Regularized Linear Regression model using Ridge Regression with $\alpha=0.001$ and comment on the differences between the three plots.

Create a pipeline for Ridge regression with polynomial features of degree 50 and scaling.

`PolynomialFeatures(degree=50)` → Adds polynomial terms up to degree 50 for each feature.

Why degree 50? → Very high degree to allow the model to capture very complex, non-linear patterns.

`StandardScaler` → Standardizes the expanded feature set to avoid numerical instability.

This is crucial because polynomial expansion can lead to large values.

`Ridge(alpha=0.001)` → Applies L2 regularization to control the model's complexity and reduce overfitting.

Small alpha allows flexibility, but still penalizes overly large coefficients.

```
[23]: pipeline_ridge = make_pipeline(PolynomialFeatures(degree=50,
    ↪ include_bias=False), StandardScaler(), Ridge(alpha=0.001))
pipeline_ridge
```

```
[23]: Pipeline(steps=[('polynomialfeatures',
    PolynomialFeatures(degree=50, include_bias=False)),
    ('standardscaler', StandardScaler()),
    ('ridge', Ridge(alpha=0.001))])
```

Compute learning curves for the ridge regression pipeline.

```
[24]: train_sizes, train_scores, valid_scores = learning_curve(pipeline_ridge, X, y,
    ↪ train_sizes=np.linspace(0.01, 1.0, 40),
    cv=5,
    ↪ scoring="neg_root_mean_squared_error")
train_errors= -train_scores.mean(axis=1)
valid_errors= -valid_scores.mean(axis=1)
```

```
[25]: print("Average training error (RMSE):", round(train_errors.mean(),3),
        "\nAverage validation error:", round(valid_errors.mean(),3))
```

Average training error (RMSE): 0.913

Average validation error: 14.08

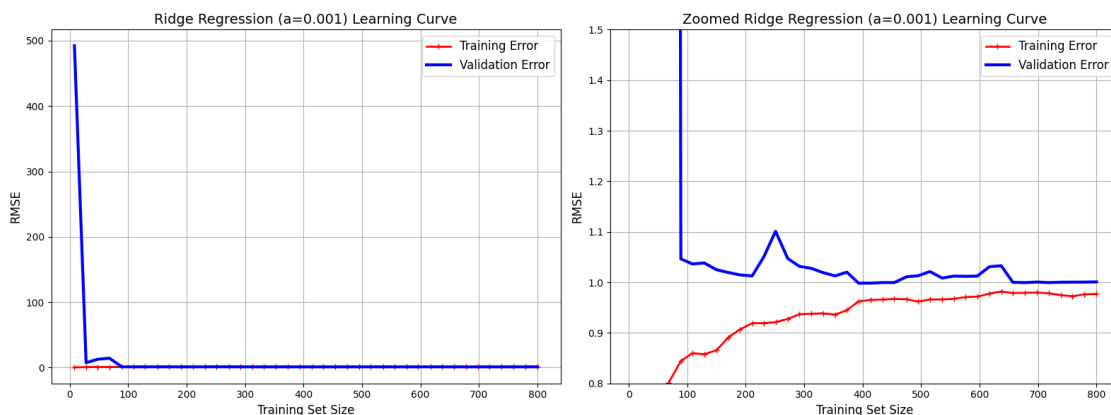
Plot the learning curves for the ridge regression model.

```
[26]: plt.figure(figsize=(16, 6))

plt.subplot(1, 2, 1)
plt.plot(train_sizes, train_errors, "r--", linewidth=2, label="Training Error")
plt.plot(train_sizes, valid_errors, "b-", linewidth=3, label="Validation Error")
plt.title("Ridge Regression (a=0.001) Learning Curve", fontsize=14)
plt.xlabel("Training Set Size", fontsize=12)
plt.ylabel("RMSE", fontsize=12)
plt.legend(loc='upper right', fontsize=12)
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(train_sizes, train_errors, "r--", linewidth=2, label="Training Error")
plt.plot(train_sizes, valid_errors, "b-", linewidth=3, label="Validation Error")
plt.title("Zoomed Ridge Regression (a=0.001) Learning Curve", fontsize=14)
plt.xlabel("Training Set Size", fontsize=12)
plt.ylabel("RMSE", fontsize=12)
plt.legend(loc='upper right', fontsize=12)
plt.ylim(0.8,1.5)
plt.grid(True)

plt.tight_layout()
plt.show()
```



Comment on Linear Regression (1.1) : Based on the learning curve, we can observe that the training error starts very low with a small training set, as the model is able to memorize the data points. As the training set size increases, the training error rises and stabilizes. The validation error starts high but quickly stabilizes as more data is used for training. However, the validation error remains fairly constant, showing that the model does not generalize better beyond a certain point. Both errors are relatively high, which indicates high bias, and end up at a plateau, fairly close to each other. This is typical of a model that is underfitting and is not complex enough to accurately capture the underlying relationship in the data

Comment on Polynomial Regression (1.2) : The learning curve reveals that the training error is nearly zero, indicating that the model is highly complex and fits the training data perfectly. However, the validation error is substantially higher, highlighting a clear gap between the training and validation performance. This behavior is characteristic of a heavily overfit model, which is commonly observed with high-degree polynomials. The model is too flexible and ends up fitting the noise in the training set, leading to poor generalization performance.

Comment on Ridge Regression (1.3) : The ridge regression with polynomial features helps reduce overfitting by adding regularization. The training error is higher compared to polynomial regression without regularization, but the validation error is significantly lower, indicating better generalization due to the regularization provided by ridge. The regularization term penalizes large coefficients, which prevents the model from fitting the noise and reduces variance, ultimately leading to a model that generalizes better to unseen data.

1.4 1.4

1.4.1 Apply 10-fold cross-validation for the simple Linear Regression model and calculate the mean RMSE and its standard deviation.

Perform 10-fold cross-validation for the linear pipeline.

```
[27]: scores = cross_val_score(pipeline_linear, X, y, scoring =  
    ↪ "neg_root_mean_squared_error", cv=10)  
average_rmse = -scores.mean()  
std_rmse = scores.std()
```

```
[28]: print(f"Mean RMSE (Linear Regression): {average_rmse:.3f}\nStandard Deviation:  
    ↪ {std_rmse:.3f}")
```

```
Mean RMSE (Linear Regression): 27.241  
Standard Deviation: 1.089
```

1.5 1.5

1.5.1 Apply 10-fold cross-validation for the polynomial model without regularization and calculate the mean RMSE.

Perform 10-fold CV for the polynomial model without regularization.


```
[29]: scores = cross_val_score(pipeline_polynomial, X, y,
    ↪scoring="neg_root_mean_squared_error", cv=10)
    average_rmse = -scores.mean()
```

```
[19]: print(f"Mean RMSE (Polynomial Regression without Regularization): {average_rmse:
    ↪.3f}")
```

Mean RMSE (Polynomial Regression without Regularization): 1.522

1.6 1.6

1.6.1 Apply 10-fold cross-validation for the regularized model and calculate the mean RMSE.

Perform 10-fold CV for the ridge regularized model.

```
[20]: scores = cross_val_score(pipeline_ridge, X, y,
    ↪scoring="neg_root_mean_squared_error", cv=10)
    average_rmse = -scores.mean()
```

```
[21]: print(f"Mean RMSE (Regularized Ridge Regression): {average_rmse:.3f}")
```

Mean RMSE (Regularized Ridge Regression): 0.994

The linear regression has a high RMSE which indicates that the model underfits the data, and with a low standard deviation, it consistently remains in the high error, reflecting the model's inability to generalize well to the data's complexity.

Introducing the polynomial features of degree 50, greatly reduces the RMSE, and the model can now capture the non linear relationships in the data. However the lack of regularization might make the model sensitive to noise in the data, and the high degree of the polynomial bears the risk of not generalizing well to new data.

Applying Ridge regularization further reduces the RMSE and now achieves the best performance among the three models. Ridge regression penalizes large coefficients (excessive complexity), helping prevent overfitting.